

Dart Programming

Lesson 8:

Applying Interactivity



SETTING UP GESTUREDETECTOR: THE BASICS

- The **GestureDetector** widget detects gestures such as tap, double tap, long press, pan, vertical drag, horizontal drag, and scale.
- It has an optional child property, and if a child widget is specified, the gestures apply only to the child widget.
- If the child widget is omitted, then the **GestureDetector** fills the entire parent instead.
- If you need to catch vertical drag and horizontal drag at the same time, use the **pan gesture**.
- If you need to catch a single-axis drag, then use either the **vertical drag** or **horizontal drag** gesture.

SETTING UP GESTUREDETECTOR: THE BASICS

- If you try to use **vertical** drag, **horizontal** drag, and **pan** gestures at the same time, you'll receive an Incorrect **GestureDetector** Arguments error.
- However, if you use either **vertical** or **horizontal** drag with a **pan** gesture, you'll not receive any errors.
- The reason you receive the error is that simultaneously having a **vertical** and **horizontal** drag gesture and a **pan** gesture results in the **pan** gesture being ignored since the other two (**vertical** and **horizontal** drag) will first catch all of the drags.
- You'll get the same kind of error if you try to use **vertical** drag, **horizontal** drag, and **scale** gestures at the same time.
- However, if you use either **vertical** or **horizontal** drag with a **scale** gesture, you won't receive any errors.

SETTING UP GESTUREDETECTOR: THE BASICS

- Each pan, **vertical drag**, **horizontal drag**, and **scale** property has a callback for each **start**, **update**, and **end** drag.
- Each callback has access to the details object containing values about the gesture, which is rich in information and provides the touch position.
 - For example, to check whether a user dragged on the screen from either left or right, you use the **onHorizontalDragEnd** callback that has access to the **DragEndDetails** details object.
 - You use the **details.primaryVelocity** value to check whether it's negative, 'Dragged Right to Left', or if it's positive, 'Dragged Left to Right'.
 - `onHorizontalDragEnd: (DragEndDetails details) {`
 - `print('onHorizontalDragEnd: $details');`
 - `if (details.primaryVelocity < 0) {`
 - `print('Dragged Right to Left: ${details.primaryVelocity}');`
 - `} else if (details.primaryVelocity > 0) {`
 - `print('Dragged Left to Right: ${details.primaryVelocity}');`
 - `}`
 - `},`

SETTING UP GESTUREDETECTOR: THE BASICS

- The following are the **GestureDetector** gestures that you can listen for and take appropriate action:
 - Tap
 - onTapDown
 - onTapUp
 - onTap
 - onTapCancel
 - Double tap
 - onDoubleTap
 - Long press
 - onLongPress
 - Pan
 - onPanStart
 - onPanUpdate
 - onPanEnd

SETTING UP GESTUREDETECTOR: THE BASICS

- Vertical drag
 - onVerticalDragStart
 - onVerticalDragUpdate
 - onVerticalDragEnd
- Horizontal drag
 - onHorizontalDragStart
 - onHorizontalDragUpdate
 - onHorizontalDragEnd
- Scale
 - onScaleStart
 - onScaleUpdate
 - onScaleEnd

Creating the Gesture, Drag-and-Drop App

- In this example, the Column widget will vertically display a **GestureDetector** listening for the **onTap**, **onDoubleTap**, **onLongPress**, and **onPanUpdate** gestures.
- It will also display a Draggable widget and a **DragTarget** widget.
- The Draggable Icon will pass a Color to the **DragTarget** displaying a Text widget changing to the passed Color.

Creating the Gesture, Drag-and-Drop App

- Create a new Flutter project for this project, you need to create only
- the pages folder.
- Open the **main.dart** file. Change the **primarySwatch** property from blue to lightGreen.
 - `primarySwatch: Colors.lightGreen,`
- Open the **home.dart** file and add to the body a **SafeArea** with a **SingleChildScrollView** as a child.
- The reason for using a **SingleChildScrollView** is to handle device rotation and automatically be able to scroll to view hidden content.
- Add a Column as a child of the **SingleChildScrollView**.
- For the Column children list of Widget, add a method call to `_buildGestureDetector()`, `_buildDraggable()`, and `_buildDragTarget()` with **Divider** widgets between them.

Creating the Gesture, Drag-and-Drop App

- body: SafeArea(
 - child: SingleChildScrollView(
 - child: Column(
 - children: <Widget>[
 - _buildGestureDetector(),
 - Divider(
 - color: Colors.black,
 - height: 44.0,
 -),
 - _buildDraggable(),
 - Divider(
 - height: 40.0,
 -),
 - _buildDragTarget(),
 - Divider(
 - color: Colors.black,
 -),
 -],
 -),
 -),
 -)

Creating the Gesture, Drag-and-Drop App

- Add the **_buildGestureDetector()** **GestureDetector** method after the Widget **build(BuildContext context) {...}**.
- Return a **GestureDetector** listening to the **onTap**, **onDoubleTap**, **onLongPress**, and
- **onPanUpdate** gestures.
- To view captured gestures, add a Container as a child of the **GestureDetector**.
- The Container child is a Column displaying an Icon and a Text widget showing the gesture detected and pointer location on the screen.
- Remember when using the **Pan** gesture that you can listen to only **onHorizontalDragUpdate** or **onVerticalDragUpdate**, not both, or you will receive an error.
- To update the screen with the pointer location and to have code reuse, create the **_displayGestureDefected(String gesture)** method.
- Each gesture passes the String representation of the gesture.
- The **onPanUpdate**, **onVerticalDragUpdate**, and **onHorizontalDragUpdate** gestures (properties) listen to **DragUpdateDetails**.

Creating the Gesture, Drag-and-Drop App

- Add the **_buildGestureDetector()** **GestureDetector** method after the **Widget build(BuildContext context) {...}**.
- Return a **GestureDetector** listening to the **onTap**, **onDoubleTap**, **onLongPress**, and
- **onPanUpdate** gestures.
- To view captured gestures, add a Container as a child of the **GestureDetector**.
- The Container child is a Column displaying an Icon and a Text widget showing the gesture detected and pointer location on the screen.
- Remember when using the **Pan** gesture that you can listen to only **onHorizontalDragUpdate** or **onVerticalDragUpdate**, not both, or you will receive an error.
- To update the screen with the pointer location and to have code reuse, create the **_displayGestureDefected(String gesture)** method.
- Each gesture passes the String representation of the gesture.
- The **onPanUpdate**, **onVerticalDragUpdate**, and **onHorizontalDragUpdate** gestures (properties) listen to **DragUpdateDetails**.

Creating the Gesture, Drag-and-Drop App

- GestureDetector _buildGestureDetector() {
- return GestureDetector(
- onTap: () {
- print('onTap');
- _displayGestureDetected('onTap');
- },
- onDoubleTap: () {
- print('onDoubleTap');
- Setting Up GestureDetector: The Basics | 273
- _displayGestureDetected('onDoubleTap');
- },
- onLongPress: () {
- print('onLongPress');
- _displayGestureDetected('onLongPress');
- },
- onPanUpdate: (DragUpdateDetails details) {
- print('onPanUpdate: \$details');
- _displayGestureDetected('onPanUpdate: \n\$details');
- },

Creating the Gesture, Drag-and-Drop App

- child: Container(
 - color: Colors.lightGreen.shade100,
 - width: double.infinity,
 - padding: EdgeInsets.all(24.0),
 - child: Column(
 - children: <Widget>[
 - Icon(
 - Icons.access_alarm,
 - size: 98.0,
 -),
 - Text('\$gestureDetected'),
 -],
 -),
 -),
 -);
 - }
 - void _displayGestureDetected(String gesture) {
 - setState() {
 - _gestureDetected = gesture;
 - };
 - }

IMPLEMENTING THE DRAGGABLE AND DRAGTARGET WIDGETS

- To implement a drag-and-drop feature, you drag the **Draggable** widget to a **DragTarget** widget.
- You use the **data** property to pass any custom data and use the **child** property to display a widget like an Icon and remains visible while not being dragged as long as the **childWhenDragging** property is **null**.
- Set the **childWhenDragging** property to display a widget while dragging.
- Use the **feedback** property to display a widget showing the user visual feedback where the widget is being dragged.
- Once the user lifts their finger on top of the **DragTarget**, the target can accept the data.
- To reject accepting the data, the user moves away from the **DragTarget** without releasing touch.
- If you need to restrict the dragging vertically or horizontally, you optionally set the **Draggable axis** property.

IMPLEMENTING THE DRAGGABLE AND DRAGTARGET WIDGETS

- To catch a single axis drag, you set the axis property to either **Axis.vertical** or **Axis.horizontal**.
- The **DragTarget** widget listens for a **Draggable** widget and receives data if dropped.
- The **DragTarget** builder property accepts three parameters: the **BuildContext**, **List<dynamic>** of acceptedData (candidateData), and **List<dynamic>** of rejectedData.
- The **acceptedData** is the data passed from the **Draggable** widget, and it expects it to be a List of values.
- The rejectedData contains the List of data that will not be accepted.

IMPLEMENTING THE DRAGGABLE AND DRAGTARGET WIDGETS

- you're going to add to the previous app an additional drag area that catches drag events.
- You'll create two widgets: a palette Icon widget that is draggable around the screen and a Text widget that receives data by accepting a drag gesture.
- When the Text widget receives data, it will change its text color from a light gray to a deep orange color but only if the drag is released on top of it.
- To keep the example clean, the Draggable data property passes the deep color orange as an integer value.
- The DragTarget accepting an integer value checks whether a Draggable is over it.
- If not, a default label shows the message "Drag To and see the color change."
- If Draggable is over it and data has a value, you see a label with the data passed.
- The ternary operator (conditional statement) is used to check whether the data is passed.

IMPLEMENTING THE DRAGGABLE AND DRAGTARGET WIDGETS

- Create the **_buildDraggable()** method, which returns a Draggable integer.
- The Draggable child is a Column with the children list of Widget consisting of an Icon and a Text widget.
- The **feedback** property is an Icon, and the data property passes the Color as an integer value.
- The **data** property can be any custom data needed, but to keep it simple, you are passing the integer value of the Color.
 - `Draggable<int> _buildDraggable() {`
 - `return Draggable(`
 - `child: Column(`
 - `children: <Widget>[`
 - `Icon(`
 - `Icons.palette,`
 - `color: Colors.deepOrange,`
 - `size: 48.0,`
 - `),`
 - `Text(`
 - `'Drag Me below to change color',`
 - `),`

IMPLEMENTING THE DRAGGABLE AND DRAGTARGET WIDGETS

-],
-),
- childWhenDragging: Icon(
 - Icons.palette,
 - color: Colors.grey,
 - size: 48.0,
-),
- feedback: Icon(
 - Icons.brush,
 - color: Colors.deepOrange,
 - size: 80.0,
-),
- data: Colors.deepOrange.value,
-);
- }

IMPLEMENTING THE DRAGGABLE AND DRAGTARGET WIDGETS

- Create the **_buildDragTarget()** method, which returns a **DragTarget** integer.
- To accept data, set the **DragTarget onAccept** property value to **colorValue** and set the **_paintedColor** variable to the **Color(colorValue)**.
- The **Color(colorValue)** constructs (converts) the integer value to a color.
- Set the **builder** property to accept three parameters: **BuildContext**, **List<dynamic> of acceptedData**, and **List<dynamic> of rejectedData**.
- Note that the data is a List, and to obtain the Color integer value, you read the first-row value by using **acceptedData[0]**.
- Use the arrow syntax with the ternary operator to check for **acceptedData.isEmpty**.
- If it's empty, it means no dragging over the **DragTarget** and shows a Text widget instructing the user to "Drag To and see color change."
- Otherwise, if there is dragging over, the **DragTarget** shows the Text widget displaying the 'Painting Color: \$acceptedData'.
- For the same Text widget, set the **color style** property to the **Color(acceptedData[0])** giving the user visual feedback of the color.

IMPLEMENTING THE DRAGGABLE AND DRAGTARGET WIDGETS

```
• DragTarget<int> _buildDragTarget() {  
•   return DragTarget<int>(  
•     onAccept: (colorValue) {  
•       _paintedColor = Color(colorValue);  
•     },  
•     builder: (BuildContext context, List<dynamic> acceptedData, List<dynamic>  
•       rejectedData) => acceptedData.isEmpty  
•       ? Text(  
•         'Drag To and see color change',  
•         style: TextStyle(color: _paintedColor),  
•       )  
•       : Text(  
•         'Painting Color: $acceptedData',  
•         style: TextStyle(  
•           color: Color(acceptedData[0]),  
•           fontWeight: FontWeight.bold,  
•         ),  
•       ),  
•     );  
•   }  
• }
```


USING THE GESTUREDETECTOR FOR MOVING AND SCALING

- The **GestureDetector** gives you the ability to implement multitouch scaling of an image by zooming in/out, double tap to increase zoom, and long press to reset the image to original size by using **onScaleStart** and **onScaleUpdate**.
- Use **onDoubleTap** to increase the zoom, and use **onLongPress** to reset the zoom to the original default size.
- When the user taps the image, the image can be dragged around to change position or scaled by zooming in/out. To accomplish both requirements, you'll use the **Transform** widget.
- Use the **Transform.scale** constructor to resize the image, and use the **Transform.translate** constructor to move the image.
- The **Transform** widget applies a transformation before the child is painted.
- Using the **Transform** default constructor, the transform argument is set by using the **Matrix4** (4D Matrix) class, and this transformation matrix is applied to the child during painting.
- The benefits of using the default constructor are to use the **Matrix4** to execute multiple cascading (**..scale()****..translate()**) transformations.
- The double dots (**..**) are used to cascade multiple transformations.

USING THE GESTUREDETECTOR FOR MOVING AND SCALING

- The **Transform** widget has four different constructors.
 - **Transform**: Default constructor taking a **Matrix4** for the transform argument.
 - **Transform.rotate**: Constructor to rotate a child widget around the center by using an angle. The angle argument rotates clockwise by radians. To rotate counterclockwise, pass a negative radian.
 - **Transform.scale**: Constructor to evenly scale a child widget on the x-axis and y-axis. The widget is scaled by its center alignment. The scale argument value of 1.0 is the original widget size. Any values above 1.0 scale the widget larger, and values below 1.0 scale the widget smaller. A value of 0.0 makes the widget invisible.
 - **Transform.translate**: Constructor to move/position a child widget by using a translation, an offset. The offset argument takes the `Offset(double dx, double dy)` class by positioning the widget on the x-axis and y-axis.

Creating an App with Gestures for Moving and Scaling

- This example is the first part of the app where you lay out the widgets that handle the gestures moving and scaling the image.
- In the next exercise, you'll concentrate on adding the logic to handle the calculations necessary to keep track of the location and scale of the image.
- The **GestureDetector** is the **body** (property) base widget and is listening for the **onScaleStart**, **onScaleUpdate**, **onDoubleTap**, and **onLongPress** gestures (properties).
- The **GestureDetector** child is a Stack that shows the image and a gesture status bar display.
- To apply the moving and scaling of the image, use the Transform widget.
- To show different ways to apply changes to a widget, you make use of three different Transform constructors: default, scale, and translate.

Creating an App with Gestures for Moving and Scaling

- Create a new Flutter project for this project, you need to create only the pages and assets/images folders.
- Create the Home Class as a **StatefulWidget** since the data (state) requires changes.
- Open the **pubspec.yaml** file to add resources. Under the assets section, add the **assets/ images/ folder**.
- Open the Terminal window (located at the bottom of your editor) and type **flutter packages get**.
- Add the folder assets and subfolder images at the project's root, and then copy the image file to the images folder.
- Open the **home.dart** file and add to the body a call to the **_buildBody(context)** method.
- The context is received by the **_buildBody()** method as a parameter for the method's MediaQuery to obtain the device width.
 - body: **_buildBody(context)**,

Creating an App with Gestures for Moving and Scaling

- Below the class `_HomeState` extends `State<Home>` and above `@override`, add the variables for `_startLastOffset`, `_lastOffset`, `_currentOffset`, `_lastScale`, and `_currentScale`.
- The `_startLastOffset`, `_lastOffset`, and `_currentOffset` variables are of type `Offset` initialized to a value of `Offset.zero`.
- The `Offset.zero` is the same as `Offset(0.0, 0.0)`, meaning the default position for the image.
- These variables are used to keep track of the image position while it's dragged.
- The `_lastScale` and `_currentScale` variables are of type `double`. They are initialized to a value of 1.0, which is normal zoom size.
- These variables are used to keep track of the image while it's scaled.
- Values greater than 1.0 scale the image bigger, and values less than 1.0 scale the image smaller.

Creating an App with Gestures for Moving and Scaling

- `class _HomeState extends State<Home> {`
- `Offset _startLastOffset = Offset.zero;`
- `Offset _lastOffset = Offset.zero;`
- `Offset _currentOffset = Offset.zero;`
- `double _lastScale = 1.0;`
- `double _currentScale = 1.0;`
- `@override`
- `Widget build(BuildContext context) {`

Creating an App with Gestures for Moving and Scaling

- Add the `_buildBody(BuildContext context)` Widget method after the Widget `build(BuildContext context) {...}`.
- Return a **GestureDetector** with the child as a Stack.
- Note that the **GestureDetector** is at the root of the body property to intercept all gestures anywhere on the screen.
- To show the image and the top gesture status bar display, you'll use the Stack widget.
- Set the Stack `fit` property to **StackFit.expand** to expand to the biggest size allowed.
- For the **Stack** children list of Widget, add three methods and name them `_transformScaleAndTranslate()`, `_transformMatrix4()`, and `_positionedStatusBar(context)`.
- The `_positionedStatusBar` method passes the context for the **MediaQuery** to obtain the full width of the device.
- Comment out the `_transformMatrix4()` method since you will be testing with the `_transformScaleAndTranslate()` first.

Creating an App with Gestures for Moving and Scaling

- Add to the **GestureDetector** the **onScaleStart**, **onScaleUpdate**, **onDoubleTap**, and **onLongPress** gestures (properties) to listen for each gesture.
- Respectively pass the **onScaleStart**, **_onScaleUpdate**, **_onDoubleTap**, and **_onLongPress** methods.
 - `Widget _buildBody(BuildContext context) {`
 - `return GestureDetector(`
 - `child: Stack(`
 - `fit: StackFit.expand,`
 - `children: <Widget>[`
 - `_transformScaleAndTranslate(),`
 - `//_transformMatrix4(),`
 - `_positionedStatusBar(context),`
 - `],`
 - `),`
 - `onScaleStart: _onScaleStart,`
 - `onScaleUpdate: _onScaleUpdate,`
 - `onDoubleTap: _onDoubleTap,`
 - `onLongPress: _onLongPress,`
 - `);`
 - `}`

Creating an App with Gestures for Moving and Scaling

- In this step you implement the first technique (moving and scaling) by nesting the **scale** and **translate** constructors.
- Add the **transformScaleAndTranslate()** Transform method after Widget **build(BuildContext context) {...}**.
- Return a **Transform** by nesting the **scale** and **translate** constructors.
- For the **Transform.scale** constructor's scale argument, enter the variable **_currentScale** and set the child argument to the **Transform.translate** constructor.
- For the **Transform.translate** constructor's offset argument, enter the variable **_current_offset** and set the child argument to an Image.
- This child widget is the image that is dragged around and scaled by nesting the two Transform widgets.
 - Transform _transformScaleAndTranslate() {
 - return Transform.scale(
 - scale: _currentScale,
 - child: Transform.translate(
 - offset: _currentOffset,
 - child: Image(
 - image: AssetImage('assets/images/elephant.jpg'),
 -),
 -),
 -);}

Creating an App with Gestures for Moving and Scaling

- In this step, you implement the second technique (moving and scaling) by using the default constructor.
- Add the `_transformMatrix4()` Transform method.
- Return a **Transform** by using the default constructor.
- For the **Transform** constructor's transform argument, use the `Matrix4`.
- Using the `Matrix4.identity()` creates the matrix from zero and sets default values.
- It is actually making a call to `Matrix4.zero()..setIdentity()`.
- From the identity constructor, use the double dots to cascade the **scale** and **translate** transformations.
- For the **scale** method, pass the `_currentScale` for both the x- and y-axes.
- The x-axis is mandatory, but the y-axis is optional.
- Since the image is scaled proportionally, both the x-axis and y-axis values are utilized.
- For the translate method, pass the `_currentOffset.dx` for the x-axis and `_currentOffset.dy` for the y-axis.
- The x-axis is mandatory, but the y-axis is optional. In this app, the image is being dragged (moved) without restrictions, and both the x-axis and y-axis values are utilized.

Creating an App with Gestures for Moving and Scaling

- To keep the image center-aligned during scaling, set the alignment property to use the **FractionalOffset.center**.
- If the center alignment is not used while the image scales, the **translate _currentOffset** moves the image.
- By keeping the image in the same location during scaling, it creates a great UX.
- If the image is moving away from the current location during scaling, that would not be a good UX.
- Set the child property to an Image widget. The image property uses the AssetImage.
 - Transform _transformMatrix4() {
 - return Transform(
 - transform: Matrix4.identity()
 - ..scale(_currentScale, _currentScale)
 - ..translate(_currentOffset.dx, _currentOffset.dy),
 - alignment: FractionalOffset.center,
 - child: Image(
 - image: AssetImage('assets/images/elephant.jpg'),
 -),
 -);
 - }

Creating an App with Gestures for Moving and Scaling

- Add the `_positionedStatusBar(BuildContext context)` Positioned method.
- Return a **Positioned** by using the default constructor. The purpose of this Positioned widget is to show a gesture status bar display on the top of the screen with the current scale and position.
- Set the **top** property to 0.0 to position it on the top of the screen in the **Stack** widget.
- Set the **width** property by using the **MediaQuery width** to expand the full width of the device.
- The child is a Container with the **color** property set to a shade of **Colors.white54**.
- Set the Container **height** property to 50.0.
- Set the Container child to a **Row** with the **mainAxisAlignment** of **MainAxisAlignment.spaceAround**.
- For the **Row** children list of Widget, use two **Text** widgets.
- The first **Text** widget shows the current scale by using the `_currentScale` variable. To show only up to four decimal points precision, use `_currentScale.toStringAsFixed(4)`.
- The second **Text** widget shows the current location by using the `_currentOffset` variable.

Creating an App with Gestures for Moving and Scaling

```
• Positioned _positionedStatusBar(BuildContext context) {  
•   return Positioned(  
•     top: 0.0,  
•     width: MediaQuery.of(context).size.width,  
•     child: Container(  
•       color: Colors.white54,  
•       height: 50.0,  
•       child: Row(  
•         mainAxisAlignment: MainAxisAlignment.spaceAround,  
•         children: <Widget>[  
•           Text(  
•             'Scale: ${_currentScale.toStringAsFixed(4)}',  
•           ),  
•           Text(  
•             'Current: $_currentOffset',  
•           ),  
•         ],  
•       ),  
•     ),  
•   );  
• }
```

Adding Logic and Calculations to the Moving and Scaling Project

- Continue with the previous moving and scaling project and start adding the logic and calculations to handle the **GestureDetector** gestures.
- The **onScaleStart** and **onScaleUpdate** are responsible for handling **moving** and **scaling** gestures.
- The **onDoubleTap** is responsible for handling the **double tapping** gesture to increase the zoom.
- The **onLongPress** is responsible for handling a **long press** gesture to reset the zoom to the original default value.

Adding Logic and Calculations to the Moving and Scaling Project

- Create the `_onScaleStart(ScaleStartDetails details)` method. This gesture is called once when the user starts to move or scale the image.
- This method populates the `_startLastOffset`, `_lastOffset`, and `_lastScale` variables, and the image position and scale calculations are based on these values.
 - `void _onScaleStart(ScaleStartDetails details) {`
 - `print('ScaleStartDetails: $details');`
 - `_startLastOffset = details.focalPoint;`
 - `_lastOffset = _currentOffset;`
 - `_lastScale = _currentScale;`
 - `}`
- Create the `_onScaleUpdate(ScaleUpdateDetails details)` method. This gesture is called when the user is either moving or scaling the image.
- By using the **details (ScaleUpdateDetails from callback)** object, different values such as the **scale**, rotation, and **focalPoint (Offset of the contact position on the device screen)** can be checked.
- The goal is to check whether the user is either moving or scaling the image by checking the **details.scale** value.
 - `void _onScaleUpdate(ScaleUpdateDetails details) {`
 - `print('ScaleUpdateDetails: $details - Scale: ${details.scale}');`
 - `}`

Adding Logic and Calculations to the Moving and Scaling Project

- Set up an if statement to check whether the user is scaling the image by evaluating **details.scale** $\neq 1.0$.
- If the **details.scale** is greater or less than 1.0, it means the image is scaling.
- With values greater than 1.0, the image is scaling larger, and with values less than 1.0, the image is scaling smaller.
 - ```
void _onScaleUpdate(ScaleUpdateDetails details) {
```
  - ```
  print('ScaleUpdateDetails: $details - Scale: ${details.scale}');
```
 - ```
 if (details.scale != 1.0) {
```
  - ```
    // Scaling
```
 - ```
 }
```
  - ```
}
```
- To calculate the current scale, create a local variable called **currentScale** of type **double** and calculate the value by taking the **_lastScale** * **details.scale**.
- The **_lastScale** was previously calculated from the **_onScaleStart()** method, and the **details.scale** is the current scale as the user continues to zoom the image.
 - ```
double currentScale = _lastScale * details.scale;
```

# Adding Logic and Calculations to the Moving and Scaling Project

- To restrict the image not to scale down to more than half its size, check whether the **currentScale** is less than 0.5 (half the original size) and reset the value to 0.5.
  - `if (currentScale < 0.5) {`
  - `currentScale = 0.5;`
  - `}`
- To have the Image widget refresh the current zoom, add the **setState()** method and populate the **\_currentScale** value with the local **currentScale** variable.
  - `setState(() {`
  - `_currentScale = currentScale;`
  - `});`
- Add an else if statement to check whether the **details.scale** is equal to 1.0.
- If the scaling is 1.0, it means the image is moving, not scaling.
  - `} else if (details.scale == 1.0) {`
- Before the image can be moved, you need to take into account the current scale since it affects the Offset (position).



# Adding Logic and Calculations to the Moving and Scaling Project

- Create the local **offsetAdjustedForScale** variable as of type **Offset**. This variable holds the last offset by taking into consideration the scale factor.
- Take the **\_startLastOffset** and subtract the **\_lastOffset**; then divide the result by the **\_lastScale**.
  - // Calculate offset depending on current Image scaling.
  - $\text{Offset offsetAdjustedForScale} = (\text{\_startLastOffset} - \text{\_lastOffset}) / \text{\_lastScale};$
- Create the local **currentOffset** variable of type **Offset**. This variable holds the current offset (position) of the image.
- The **currentOffset** is calculated by taking the **details.focalPoint** minus the **offsetAdjustedForScale** times the **\_currentScale**.
  - $\text{Offset currentOffset} = \text{details.focalPoint} - (\text{offsetAdjustedForScale} * \text{\_currentScale});$
- To have the Image widget refresh the current position, add the **setState()** method and populate the **\_currentOffset** value with the local **currentOffset** variable.
  - `setState() {`
  - `\_currentOffset = currentOffset;`
  - `});`



# Adding Logic and Calculations to the Moving and Scaling Project

- The following is the full `_onScaleUpdate()` method:
  - `void _onScaleUpdate(ScaleUpdateDetails details) {`
  - `print('ScaleUpdateDetails: $details - Scale: ${details.scale}');`
  - `if (details.scale != 1.0) {`
  - `double currentScale = _lastScale * details.scale;`
  - `if (currentScale < 0.5) {`
  - `currentScale = 0.5;}`
  - `setState() {`
  - `_currentScale = currentScale;`
  - `});`
  - `print('_scale: $_currentScale - _lastScale: $_lastScale');`
  - `} else if (details.scale == 1.0) {`
  - `Offset offsetAdjustedForScale = (_startLastOffset - _lastOffset) /`
  - `_lastScale;`
  - `Offset currentOffset = details.focalPoint - (offsetAdjustedForScale *`
  - `_currentScale);`
  - `setState() {`
  - `_currentOffset = currentOffset;`
  - `});`
  - `print('offsetAdjustedForScale: $offsetAdjustedForScale - _currentOffset:`
  - `$_currentOffset');`
  - `}}`

# Adding Logic and Calculations to the Moving and Scaling Project

- Create the **\_onDoubleTap()** method. This gesture is called when the user is double tapping the screen.
- When a double tap is detected, the image is scaled twice as large.
- Create the local **currentScale** variable of type double.
- The **currentScale** is calculated by taking the **\_lastScale** times 2.0 (twice the size).
  - `double currentScale = _lastScale * 2.0;`
- Add an if statement that checks whether the **currentScale** is greater than 16.0 (16 times the original size); then reset **currentScale** to 1.0.
- Add a call to the **\_resetToDefaultValues()** method that resets all variables to their default values.
- The result is that the image is recentered and scaled to the original size.
- After the if statement, set the **\_lastScale** variable to the local **currentScale** variable.
- To have the Image widget refresh the current scale, add the **setState()** method and populate the **\_currentScale** value with the local **currentScale** variable.

# Adding Logic and Calculations to the Moving and Scaling Project

- Create the **\_onDoubleTap()** method. This gesture is called when the user is double tapping the screen.
- When a double tap is detected, the image is scaled twice as large.
- Create the local **currentScale** variable of type double.
- The **currentScale** is calculated by taking the **\_lastScale** times 2.0 (twice the size).
  - `double currentScale = _lastScale * 2.0;`
- Add an if statement that checks whether the **currentScale** is greater than 16.0 (16 times the original size); then reset **currentScale** to 1.0.
- Add a call to the **\_resetToDefaultValues()** method that resets all variables to their default values.
- The result is that the image is recentered and scaled to the original size.
- After the if statement, set the **\_lastScale** variable to the local **currentScale** variable.
- To have the Image widget refresh the current scale, add the **setState()** method and populate the **\_currentScale** value with the local **currentScale** variable.

# Adding Logic and Calculations to the Moving and Scaling Project

- `void _onDoubleTap() {`
- `print('onDoubleTap');`
- `double currentScale = _lastScale * 2.0;`
- `if (currentScale > 16.0) {`
- `currentScale = 1.0;`
- `_resetToDefaultValues();`
- `}`
- `_lastScale = currentScale;`
- `setState(() {`
- `_currentScale = currentScale;`
- `});`
- `}`

# Adding Logic and Calculations to the Moving and Scaling Project

- Create the `_onLongPress()` method. This gesture is called when the user is pressing and holding on the screen.
- When a long press is detected, the image is reset to its original position and scale.
- To have the Image widget refresh to the default position and scale, add the `setState()` method and call the `_resetToDefaultValues()` method.
  - `void _onLongPress() {`
  - `print('onLongPress');`
  - `setState(() {`
  - `_resetToDefaultValues();`
  - `});`
  - `}`